

 UNIWERSYTET WSB MERITO POZNAŃ	Kierunek: Informatyka
Programowanie zaawansowane	Grupa: K79
Tytuł projektu: "PartyTavern"	Autorzy: Adam Horoszkiewicz Maciej Idzikowski

1. Opis projektu

PartyTavern to strona internetowa przeznaczona dla graczy. Celem aplikacji jest ułatwienie znalezienia drużyny do wspólnych aktywności w wybranych grach. Zalogowani użytkownicy będą mogli zamieszczać posty typu "Szukam drużyny" i dołączać do takich drużyn. Niezalogowane osoby będą miały dostęp tylko do przeglądania ogłoszeń. Dodatkowo użytkownicy będą mogli zaproponować dodanie nowej gry do bazy, prośby te będą akceptowane przez Administratora. Celem strony ma być integracja graczy różnych gier, tworzenie wspólnego community oraz umożliwienie graczom wygodniejsze znalezienie drużyny bez konieczności korzystania ze wbudowanych systemów LFG.

2. Technologie wykorzystane w projekcie

Projekt został zbudowany z wykorzystaniem .NET 8, bazy SQL Server oraz języka C#. Dodatkowo projekt korzysta z pakietów .NET, które znacząco przyspieszyły pracę projektowe.

3. Instrukcja pierwszego uruchomienia projektu

- Wymagania wstępne:
 - .NET SDK wersja 8.0.11
 - Visual Studio (wskazana wersja 2022 lub nowsza)
 - Uruchom plik: PartyTavern/PartyTavern.sln
- W konsoli menedżera pakietów wpisz polecenie "update-database".
- Uruchom projekt przyciskiem Start lub F5.
- Pierwsze logowanie:
 - Konto admina: admin@admin.admin:Admin123!
 - Konta użytkownika można utworzyć klikając przycisk "Rejestracja" (nie zapomnij potwierdzić emaila na stronie po rejestracji).

- Baza gier domyślnie jest pusta, więc należy zalogować się na konto administratora i dodać grę (bądź kilka) klikając przycisk Gry>Dodaj nową grę.
- **Zarządzanie drużynami:**
 - By założyć drużynę należy kliknąć Drużyny>Stwórz nową drużynę.
 - By zobaczyć szczegóły drużyny należy kliknąć przycisk szczegóły znajdujący się obok niej.
 - Opcje edytowania/usuwania/dołączania do drużyny oraz komentowania są dostępne tylko dla administratora bądź twórcy drużyny/komentarza z widoku szczegółów.
- **Propozycje gier:**
 - Użytkownik może złożyć propozycję dodania nowej gry przez przycisk “Dodaj propozycję gry!!”.
 - Administrator może zobaczyć propozycje w widoku “Propozycje gier”.

4. Opis struktury projektu

Projekt jest tworzony z wykorzystaniem wzorca MVC (Model-View-Controller), oznacza to, że w projekcie mamy 3 główne typy elementów: Model - czyli obiekt, który za pomocą mechanizmu migracji jest tworzony w bazie danych. View - czyli plik .cshtml, w którym jest zawarty kod wyświetlany na danej stronie. Controller - czyli plik .cs w którym znajduje się logika zarządzająca danymi oraz metody wykorzystywane w widokach.

5. Modele w projekcie

W projekcie mamy 5 głównych modeli:

- Comment - model Komentarza, który użytkownicy mogą zostawić pod postem drużyny.
- Game - model gier, które można wybrać przy tworzeniu postów.
- GameProposal - model propozycji, które przesyłają użytkownicy.
- TeamMember - model przechowujący dane o osobach, które dołączyły do drużyny
- TeamPost - model przechowujący informacje o postach.

Dodatkowo są również dwa modele techniczne:

- ErrorViewModel - domyślny widok techniczny
- FutureDateAttribute - model potrzebny do obsługi wyświetlania przedawnionych postów

Comment:

```
public class Comment
{
    public int Id { get; set; }

    [Required]                                     //Wymagalność
    [MaxLength(500)]                               //Maksymalna długość
    public string Content { get; set; }             //Treść komentarza
    public DateTime CreatedAt { get; set; }         //Data utworzenia komentarza
}
```

```

public string UserId { get; set; } //Id użytkownika
public virtual IdentityUser User { get; set; } //Odwołanie do użytkownika, który dodał
Komentarz
public int PostId { get; set; } //Id postu, do którego przypisany jest Komentarz
public virtual TeamPost Post { get; set; } //odwołanie do postu, do którego odnosi się
Komentarz
public bool wasEdited { get; set; } //informacja o tym, czy Komentarz był edytowany
}

```

Game:

```

public class Game
{
    public int Id { get; set; } //id gry
    public string Name { get; set; } = string.Empty; //nazwa gry
    public string Genre { get; set; } = string.Empty; //gatunek gry
    public string Description { get; set; } = string.Empty; //Opis gry
}

```

GameProposal:

```

public class GameProposal
{
    [Key]
    public int Id { get; set; } //id propozycji

    [Required]
    [MaxLength(100)]
    public required string Name { get; set; } // Nazwa gry

    [MaxLength(50)]
    public string Genre { get; set; } // Gatunek gry

    [Required]
    public required string Description { get; set; } // Opis gry
    public bool IsApproved { get; set; } // Czy propozycja została zatwierdzona
    public DateTime SubmittedDate { get; set; } = DateTime.UtcNow; // Data zgłoszenia
}

```

TeamMember:

```

public class TeamMember
{
    public int Id { get; set; } // id członka drużyny
}

```

```

public string UserId { get; set; } //id użytkownika
public IdentityUser User { get; set; } //odwołanie do użytkownika
public int TeamPostId { get; set; } //id postu do którego dołączył użytkownik
    public TeamPost TeamPost { get; set; } //odwołanie do postu, do którego
dołączył użytkownik
}

```

TeamPost:

```

public class TeamPost
{
    public int Id { get; set; }

    [Required(ErrorMessage = "Wybierz grę.")]
    [ForeignKey("Game")]
    public int GameId { get; set; }
    public Game? Game { get; set; }

    // rozmiar drużyny wraz z komunikatem błędu i wymaganiami
    [Required(ErrorMessage = "Wybierz rozmiar drużyny.")]
    [Range(2, 20, ErrorMessage = "Drużyna nie może być mniejsza niż 2 i większa niż 20.")]
    public int TeamSize { get; set; }
    public int CurrentTeamSize { get; set; }
    // opis drużyny wraz z komunikatem błędu oraz wymaganiami
    [Required]
    [StringLength(500, ErrorMessage = "Opis nie może być dłuższy niż 500 znaków.")]
    public string Description { get; set; } = string.Empty;

    // data utworzenia wraz z wymaganiami oraz komunikatem błędu
    [Required]
    [DataType(DataType.DateTime)]
    [Display(Name = "Termin gry")]
    [FutureDate(ErrorMessage = "Data gry nie może być wcześniejsza niż obecna.")]
    public DateTime NeededBy { get; set; }
    public DateTime CreatedAt { get; set; } = DateTime.Now;

    // id użytkownika
    public string UserId { get; set; } = string.Empty;
    // Powiązanie z członkami drużyny
    public ICollection<TeamMember> TeamMembers { get; set; } = new
    List<TeamMember>();
    // Powiązanie z komentarzami
    public ICollection<Comment> Comments { get; set; } = new List<Comment>();
}

```

ErrorViewModel:

```
public class ErrorViewModel
{
    public string? RequestId { get; set; }

    public bool ShowRequestId => !string.IsNullOrEmpty(RequestId);
}
```

FutureDateAttribute:

```
public class FutureDateAttribute : ValidationAttribute
{
    public override bool IsValid(object value)    //walidowanie daty w celu
    wyświetlenia posta w odpowiednim widoku
    {
        if (value is DateTime date)
        {
            return date >= DateTime.Now;
        }
        return true;
    }
}
```

6. Kontrolery wraz z metodami

W projekcie są trzy kontrolery:

- **AdminController** - do zarządzania funkcjonalnościami dostępnymi tylko dla Administratora.
- **TeamPostsController** - do zarządzania postami dotyczącymi drużyn
- **HomeController** - do zarządzania głównym widokiem i funkcjonalnością wysyłania propozycji nowych gier.

AdminController:

```
[Authorize(Roles = "Admin")]    // metody kontrolera są dostępne tylko dla roli admina
public class AdminController : Controller
{
    private readonly ApplicationDbContext _context;

    public AdminController(ApplicationDbContext context)
    {
        _context = context;
    }

    // Akcja dodawania nowej gry
```

```
public IActionResult AddGame()
{
    return View();
}

[HttpPost]

public async Task<IActionResult> AddGame(Game game)
{
    if (ModelState.IsValid)
    {
        _context.Games.Add(game);
        await _context.SaveChangesAsync();
        return RedirectToAction("Games");
    }
    return View(game);
}

// Akcja wyświetlania dostępnych gier
public async Task<IActionResult> Games()
{
    var games = await _context.Games.ToListAsync(); // Pobranie gier z bazy danych
    return View(games); // Przekazanie gier do widoku
}

// Akcja edytowania gry
public async Task<IActionResult> EditGame(int id)
{
    var game = await _context.Games.FindAsync(id); // Pobranie gry z bazy danych
    if (game == null)
    {
        return NotFound(); // Jeśli gry nie ma, zwróć błąd 404
    }
    return View(game); // Przekazanie gry do widoku edycji
}

[HttpPost]
public async Task<IActionResult> EditGame(int id, Game game)
{
    if (id != game.Id)
    {
        return NotFound(); // Jeśli id gry się nie zgadza, zwróć błąd 404
    }

    if (ModelState.IsValid)
    {
        try
```

```

    {
        _context.Update(game); // Zaktualizowanie gry w bazie danych
        await _context.SaveChangesAsync();
    }
    catch (DbUpdateConcurrencyException)
    {
        if (!_context.Games.Any(g => g.Id == id))
        {
            return NotFound(); // Jeśli gry nie ma w bazie, zwróć błąd 404
        }
        else
        {
            throw; // Inny błąd, rzuć wyjątek
        }
    }
    return RedirectToAction("Games"); // Przekierowanie na stronę z grami
}
return View(game); // Jeśli formularz jest niepoprawny, wróć do formularza
}

// Akcja usuwania gry
// Nowa akcja dla usuwania gry
[Authorize(Roles = "Admin")]
public async Task<IActionResult> DeleteGame(int id)
{
    var game = await _context.Games.FindAsync(id); // Pobranie gry z bazy danych
    if (game == null)
    {
        return NotFound(); // Jeśli gry nie ma, zwróć błąd 404
    }
    return View(game); // Przekazanie gry do widoku
}

[HttpPost, ActionName("DeleteGame")]
[Authorize(Roles = "Admin")]
public async Task<IActionResult> DeleteGame(int id, Game game)
{
    if (game == null)
    {
        return NotFound(); // Jeśli gry nie ma, zwróć błąd 404
    }

    _context.Games.Remove(game); // Usunięcie gry z bazy danych
    await _context.SaveChangesAsync();
    return RedirectToAction("Games"); // Przekierowanie na stronę z grami
}

```

```

// Wyświetlanie propozycji gier
public async Task<IActionResult> GameProposals()
{
    var proposals = await _context.GameProposals
        .ToListAsync(); // Pobranie propozycji z bazy danych
    return View(proposals); // Przekazanie propozycji do widoku
}

// Zatwierdzanie propozycji
public async Task<IActionResult> ApproveProposal(int id)
{
    var proposal = await _context.GameProposals.FindAsync(id); // Pobranie
    propozycji
    if (proposal == null)
    {
        return NotFound(); // Jeśli propozycji nie ma, zwróć błąd 404
    }

    // Tworzenie gry na podstawie propozycji
    var game = new Game
    {
        Name = proposal.Name,
        Genre = proposal.Genre,
        Description = proposal.Description
    };

    _context.Games.Add(game);
    _context.GameProposals.Remove(proposal); // Usunięcie propozycji po
    zatwierdzeniu
    await _context.SaveChangesAsync();

    return RedirectToAction("GameProposals"); // Przekierowanie na listę propozycji
}

// Odrzucanie propozycji
public async Task<IActionResult> RejectProposal(int id)
{
    var proposal = await _context.GameProposals.FindAsync(id); // Pobranie
    propozycji
    if (proposal == null)
    {
        return NotFound(); // Jeśli propozycji nie ma, zwróć błąd 404
    }

    _context.GameProposals.Remove(proposal); // Usunięcie propozycji
    await _context.SaveChangesAsync();
}

```



```
return RedirectToAction("GameProposals"); // Przekierowanie na listę propozycji
}}
```

HomeController:

```
public class HomeController : Controller
{
    private readonly ILogger<HomeController> _logger;
    private readonly UserManager<IdentityUser> _userManager;
    private readonly ApplicationDbContext _context;

    public HomeController(ILogger<HomeController> logger, UserManager<IdentityUser>
userManager, ApplicationDbContext context)
    {
        _logger = logger;
        _userManager = userManager;
        _context = context;
    }

    public async Task<IActionResult> Index()
    {
        var user = await _userManager.GetUserAsync(User); // Pobierz bieżącego
użytkownika
        bool isAdmin = user != null && await _userManager.IsInRoleAsync(user, "Admin");

        // Przekaż dane do widoku za pomocą ViewData
        ViewData["IsAdmin"] = isAdmin;

        return View();
    }

    public IActionResult Privacy()
    {
        return View();
    }

    [ResponseCache(Duration = 0, Location = ResponseCacheLocation.None, NoStore =
true)]
    public IActionResult Error()
    {
        return View(new ErrorViewModel { RequestId = Activity.Current?.Id ??
HttpContext.TraceIdentifier });
    }

    public IActionResult CreateProposal()
```

```

{
    return View();
}

[HttpPost]
// Metoda
public async Task<IActionResult> CreateProposal(GameProposal proposal)
{
    if (ModelState.IsValid)
    {
        _context.GameProposals.Add(proposal);
        await _context.SaveChangesAsync();
        return RedirectToAction("ProposalSubmitted");
    }

    return View(proposal);
}

// Akcja potwierdzająca zgłoszenie propozycji
public IActionResult ProposalSubmitted()
{
    return View();
}

```

TeamPostController:

```

public class TeamPostsController : Controller
{
    private readonly ApplicationDbContext _context;
    private readonly UserManager<IdentityUser> _userManager;

    public TeamPostsController(ApplicationDbContext context, UserManager<IdentityUser>
userManager)
    {
        _context = context;
        _userManager = userManager;
    }

    // GET: /TeamPosts/Index
    public async Task<IActionResult> Index(bool showExpired = false)
    {
        var userId = _userManager.GetUserId(User); // Uzyskaj UserId aktualnie
zalogowanego użytkownika
    }
}

```

```

IEnumerable<TeamPost> query;

if (showExpired)
{
    // Pobierz posty z przeszłości
    query = _context.TeamPosts
        .Include(p => p.Game)
        .Include(p => p.TeamMembers)
        .ThenInclude(tm => tm.User)
        .Where(p => p.NeededBy < DateTime.Now); // Posty przedawnione
}
else
{
    // Pobierz aktywne posty
    query = _context.TeamPosts
        .Include(p => p.Game)
        .Include(p => p.TeamMembers)
        .ThenInclude(tm => tm.User)
        .Where(p => p.NeededBy >= DateTime.Now); // Posty w przyszłości
}

var posts = await query.ToListAsync();

// Przekazujemy ID użytkownika i czy pokazujemy przedawnione posty
ViewData["CurrentUserId"] = userId;
ViewData["ShowExpired"] = showExpired;

return View(posts);
}

// GET: /TeamPosts/Create
public IActionResult Create()
{
    ViewBag.Games = _context.Games.ToList(); // Lista gier do wyboru
    return View();
}

// POST: /TeamPosts/Create
[HttpPost]
public async Task<IActionResult> Create(TeamPost post)
{
    if (ModelState.IsValid)
    {
        post.UserId = _userManager.GetUserId(User);
        post.CreatedAt = DateTime.Now;
    }
}

```

```

        Console.WriteLine($"Current User ID: {_userManager.GetUserId(User)}");
        Console.WriteLine($"User ID: {post.UserId}");

        // Dodaj członka do drużyny
        post.TeamMembers.Add(new TeamMember
        {
            UserId = post.UserId,
            TeamPostId = post.Id
        });
        post.CurrentTeamSize++;

        // Dodaj post do bazy danych
        _context.TeamPosts.Add(post);
        await _context.SaveChangesAsync();

        return RedirectToAction("Index");
    }

    ViewBag.Games = _context.Games.ToList();
    return View(post);
}

[HttpPost]
[Authorize]
public async Task<ActionResult> Join(int postId)
{
    var teamPost = await _context.TeamPosts
        .Include(p => p.TeamMembers)
        .FirstOrDefaultAsync(p => p.Id == postId);

    if (teamPost == null)
    {
        return NotFound();
    }

    // Sprawdź, czy drużyna jest przedawniona
    if (teamPost.NeededBy < DateTime.Now)
    {
        ModelState.AddModelError(string.Empty, "Nie można dołączyć do przedawnionej drużyny.");
        return RedirectToAction("Index");
    }

    var userId = _userManager.GetUserId(User);

    // Sprawdź, czy użytkownik już dołączył do tej drużyny
    var existingMember = await _context.TeamMembers

```

```

        .FirstOrDefaultAsync(tm => tm.TeamPostId == postId && tm.UserId == userId);

    if (existingMember != null)
    {
        ModelState.AddModelError(string.Empty, "You have already joined this team.");
        return RedirectToAction("Index");
    }

    // Sprawdź, czy drużyna nie jest pełna
    if (teamPost.CurrentTeamSize >= teamPost.TeamSize)
    {
        ModelState.AddModelError(string.Empty, "The team is already full.");
        return RedirectToAction("Index");
    }

    // Dodaj członka do drużyny
    _context.TeamMembers.Add(new TeamMember
    {
        UserId = userId,
        TeamPostId = postId
    });

    // Zaktualizuj rozmiar drużyny
    teamPost.CurrentTeamSize++;

    await _context.SaveChangesAsync();

    return RedirectToAction("Index");
}

[HttpPost]
[Authorize]
public async Task<IActionResult> Leave(int postId)
{
    var currentUserId = _userManager.GetUserId(User);

    var post = await _context.TeamPosts
        .Include(p => p.TeamMembers)
        .FirstOrDefaultAsync(p => p.Id == postId);

    if (post == null)
    {
        return NotFound();
    }

    // Sprawdzamy, czy użytkownik jest twórcą drużyny
    if (post.UserId == currentUserId)

```

```

    {
        TempData["Error"] = "Twórca drużyny nie może jej opuścić.";
        return RedirectToAction("Details", new { id = postId });
    }

    var teamMember = post.TeamMembers.FirstOrDefault(tm => tm.UserId ==
currentUserId);
    if (teamMember != null)
    {
        _context.TeamMembers.Remove(teamMember);
        post.CurrentTeamSize--;
        await _context.SaveChangesAsync();

        TempData["Message"] = "Pomyślnie opuściłeś drużynę.";
    }
    else
    {
        TempData["Error"] = "Nie jesteś członkiem tej drużyny.";
    }

    return RedirectToAction("Details", new { id = postId });
}

public async Task<ActionResult> Details(int id, string returnUrl = null)
{
    var post = await _context.TeamPosts
        .Include(p => p.Game)
        .Include(p => p.TeamMembers).ThenInclude(tm => tm.User)
        .Include(p => p.Comments).ThenInclude(c => c.User)
        .FirstOrDefaultAsync(p => p.Id == id);

    if (post == null)
    {
        return NotFound();
    }

    ViewData["CurrentUserId"] = _userManager.GetUserId(User);
    ViewData["ReturnUrl"] = returnUrl ?? Url.Action("Index", "TeamPosts");
    return View(post);
}

public async Task<ActionResult> Edit(int id)
{
    var post = await _context.TeamPosts
        .Include(p => p.Game) // Załaduj powiązaną grę
        .FirstOrDefaultAsync(p => p.Id == id);

```

```

    if (post == null)
    {
        return NotFound();
    }

    // Pobierz listę gier do wyboru w formularzu
    ViewBag.Games = new SelectList(await _context.Games.ToListAsync(), "Id",
    "Name");

    // Sprawdzamy, czy użytkownik jest administratorem lub twórcą posta
    var userId = _userManager.GetUserId(User);
    if (post.UserId != userId && !User.IsInRole("Admin"))
    {
        return Unauthorized(); // Jeśli nie jest twórcą ani adminem, zwróć błąd
    }

    // Przekaż post do widoku
    return View(post);
}

// POST: /TeamPosts/Edit/{id}
[HttpPost]
[ValidateAntiForgeryToken]
public async Task<ActionResult> Edit(int id, TeamPost post)
{
    if (id != post.Id)
    {
        return NotFound();
    }

    // Pobierz drużynę z bazy danych
    var teamPost = await _context.TeamPosts
        .Include(p => p.TeamMembers) // Wczytaj członków drużyny
        .FirstOrDefaultAsync(p => p.Id == id);

    if (teamPost == null)
    {
        return NotFound();
    }

    // Sprawdzenie, czy proponowany rozmiar drużyny jest wystarczający
    if (post.TeamSize < teamPost.TeamMembers.Count)
    {
        ModelState.AddModelError("TeamSize", "Rozmiar drużyny nie może być mniejszy
        niż liczba zapisanych członków.");
        return View(post); // Zwróć formularz z błędem
    }
}

```

```

teamPost.TeamSize = post.TeamSize;

// Zaktualizuj inne właściwości drużyny
teamPost.Description = post.Description;
teamPost.NeededBy = post.NeededBy;
teamPost.GameId = post.GameId;

// Zaktualizuj drużynę w bazie danych
_context.Update(teamPost);
await _context.SaveChangesAsync();

return RedirectToAction(nameof(Index));
}

// GET: /TeamPosts/Delete/{id}
public async Task<IActionResult> Delete(int id)
{
    var post = await _context.TeamPosts.FindAsync(id);
    if (post == null)
    {
        return NotFound();
    }

    // Sprawdzamy, czy użytkownik jest administratorem lub twórcą posta
    var userId = _userManager.GetUserId(User);
    if (post.UserId != userId && !User.IsInRole("Admin"))
    {
        return Unauthorized(); // Jeśli nie jest twórcą ani adminem, zwróć błąd
    }

    return View(post);
}

// POST: /TeamPosts/Delete/{id}
[HttpPost, ActionName("Delete")]
public async Task<IActionResult> Delete(int id, TeamPost post)
{
    if (post == null)
    {
        return NotFound();
    }

    var userId = _userManager.GetUserId(User);
    Console.WriteLine($"Current User ID: {userId}");
    Console.WriteLine($"Post User ID: {post.UserId}");

```



```

        if (post.UserId != userId && !User.IsInRole("Admin"))
        {
            Console.WriteLine("Unauthorized access attempt.");
            return Unauthorized();
        }

        _context.TeamPosts.Remove(post);
        await _context.SaveChangesAsync();
        return RedirectToAction("Index");
    }

    [Authorize]
    public async Task<ActionResult> MyTeams(bool showExpired = false)
    {
        var currentUserId = User.FindFirstValue(ClaimTypes.NameIdentifier);

        IQueryable<TeamPost> teamPostsQuery = _context.TeamPosts
            .Where(p => p.TeamMembers.Any(tm => tm.UserId == currentUserId));

        if (!showExpired)
        {
            teamPostsQuery = teamPostsQuery.Where(p => p.NeededBy >= DateTime.Now);
            // Pokaż tylko aktywne drużyny
        }
        else
        {
            teamPostsQuery = teamPostsQuery.Where(p => p.NeededBy < DateTime.Now); //
            Pokaż tylko przedawnione drużyny
        }

        var teamPosts = await teamPostsQuery
            .Include(p => p.Game)
            .Include(p => p.TeamMembers).ThenInclude(tm => tm.User)
            .ToListAsync();

        ViewData["ShowExpired"] = showExpired; // Ustawiamy wartość dla widoku

        return View(teamPosts);
    }

    // Akcja do dodawania komentarza
    [HttpPost]
    [Authorize]
    [ValidateAntiForgeryToken]
    public IActionResult AddComment(int postId, string content)
    {

```

```

var post = _context.TeamPosts.Find(postId);

if (post == null)
{
    return NotFound();
}

if (post.NeededBy < DateTime.Now)
{
    TempData["ErrorMessage"] = "Nie można dodawać komentarzy do
przedawnionych postów.";
    return RedirectToAction("Details", new { id = postId });
}

if (string.IsNullOrEmpty(content))
{
    TempData["ErrorMessage"] = "Komentarz nie może być pusty.";
    return RedirectToAction("Details", new { id = postId });
}

var currentUserId = _userManager.GetUserId(User);

var comment = new Comment
{
    Content = content,
    CreatedAt = DateTime.Now,
    UserId = currentUserId,
    PostId = postId
};

_context.Comments.Add(comment);
_context.SaveChanges();

return RedirectToAction("Details", new { id = postId });
}

[HttpPost]
[Authorize]
[ValidateAntiForgeryToken]
public async Task<IActionResult> DeleteComment(int commentId, int postId)
{
    var comment = await _context.Comments
        .Include(c => c.User)
        .FirstOrDefaultAsync(c => c.Id == commentId);

    if (comment == null)

```

```

    {
        return NotFound();
    }

    var currentUserId = _userManager.GetUserId(User);

    // Sprawdź, czy użytkownik jest właścicielem komentarza lub adminem
    if (comment.UserId != currentUserId && !User.IsInRole("Admin"))
    {
        return Forbid();
    }

    _context.Comments.Remove(comment);
    await _context.SaveChangesAsync();

    return RedirectToAction("Details", new { id = postId });
}

[HttpGet]
[Authorize]
public async Task<IActionResult> EditComment(int commentId)
{
    var comment = await _context.Comments
        .Include(c => c.User)
        .FirstOrDefaultAsync(c => c.Id == commentId);

    if (comment == null)
    {
        return NotFound();
    }

    var currentUserId = _userManager.GetUserId(User);

    // Sprawdź, czy użytkownik jest właścicielem komentarza
    if (comment.UserId != currentUserId)
    {
        return Forbid();
    }

    return View(comment);
}

[HttpPost]
[Authorize]
[ValidateAntiForgeryToken]
public async Task<IActionResult> EditComment(int commentId, string content)
{

```

```

var comment = await _context.Comments.FindAsync(commentId);

if (comment == null)
{
    return NotFound();
}

var currentUserId = _userManager.GetUserId(User);

// Sprawdź, czy użytkownik jest właścicielem komentarza
if (comment.UserId != currentUserId)
{
    return Forbid();
}

if (string.IsNullOrEmpty(content))
{
    ModelState.AddModelError("", "Komentarz nie może być pusty.");
    return View(comment);
}

comment.Content = content;
comment.wasEdited = true;

_context.Comments.Update(comment);
await _context.SaveChangesAsync();

return RedirectToAction("Details", new { id = comment.PostId });
}

private bool TeamPostExists(int id)
{
    return _context.TeamPosts.Any(e => e.Id == id);
}

```

7. System użytkowników

System użytkowników w projekcie jest oparty o pakiet Microsoft.EntityFrameworkCore, dzięki czemu jest prosty i przejrzysty. Część funkcjonalności jest dostępna tylko dla zalogowanych użytkowników. Dodatkowo w systemie istnieje również rola administratora, która pozwala dodawać nowe gry i akceptować prośby o dodanie nowych gier. Dodatkowo może moderować wszystkie treści. W celu nadania kontu uprawnień administratora należy wpisać mail przypisany do tego kontu w pliku Program.cs w linii 62. Domyślnym kontem Administratora jest [“admin@admin.admin”](mailto:admin@admin.admin).

8. Ciekawe funkcjonalności

Zdecydowanie najciekawszą funkcjonalnością jest **automatyczne dodawanie gry do bazy gier na podstawie propozycji użytkowników**. Administrator może takie propozycje odrzucić lub zaakceptować, a gdy je zaakceptuje, to gra jest samoistnie dodawana do listy gier.

Kolejną wartą uwagi funkcjonalnością jest **możliwość komentowania ogłoszeń o drużynach**. Jest to prosta, ale bardzo przydatna funkcjonalność, która może być szybko i sprawnie zarządzana przez Administratora.